



Assignment - Python_Dja...
PDF

With the shared docx, explain me everything about the assignment.

I'll break the assignment down into **what they are actually asking you to build, the business logic, Django expectations, deliverables, AI-usage rules, testing requirements, and what can cause rejection.** I'll distinguish what is explicitly required by the document from reasonable implementation choices.

1. What is the assignment?

The assignment is titled **"Python/Django Hiring Assignment: AI-Assisted Box Selection System."**

Assignment - Python_Django Deve...

The business scenario is an e-commerce platform:

1. A customer places an order.
2. The order contains one or more products.
3. Every product has:
 - Dimensions
 - Weight
4. The warehouse needs to determine which shipping box to use.
5. Every box has:
 - Internal dimensions
 - Maximum weight capacity
 - Cost
6. Your Django application must recommend the **most suitable box** for the order.

Assignment - Python_Django Deve...

So the core problem is essentially:

Given an order containing products, determine which available box can physically contain those products and satisfy its weight limit, then recommend the appropriate box.

The document does **not** prescribe a particular algorithm, database schema, API design, UI, or exact definition of "most suitable." Those are areas where your engineering judgment matters.

2. What they are evaluating

This is not simply a CRUD Django assignment.

The important part is that they want to see **your engineering reasoning**.

The document explicitly says AI tools are allowed, including ChatGPT, Claude, Cursor, GitHub Copilot, etc., but the submission must clearly show **your own thinking**.

Assignment - Python_Django Deve...

That means they are likely interested in:

- How you model products and boxes
- How you represent dimensions
- How you calculate whether products fit
- How you handle multiple products
- How you handle weight limits
- How you select among multiple valid boxes
- How you deal with invalid/impossible orders
- How you test the business logic
- How you structure the Django project
- Whether you understand the code you submit
- Whether you can critically evaluate AI-generated code

3. The central problem: Box selection

Imagine you have:

Products

Product	Dimensions	Weight
Laptop	30 × 20 × 3 cm	2 kg

Product	Dimensions	Weight
Keyboard	45 × 15 × 3 cm	1 kg

And boxes:

Box	Internal dimensions	Max weight	Cost
Small	50 × 30 × 10 cm	5 kg	₹50
Medium	60 × 40 × 20 cm	10 kg	₹80
Large	100 × 80 × 50 cm	20 kg	₹150

Your system needs to determine which box can accommodate the order.

There are **two separate constraints**:

Physical constraint

The products need to fit inside the box.

Weight constraint

The total product weight must not exceed the box's maximum weight.

For example:

Total order weight = 2 + 1

= 3 kg

Therefore:

Box maximum weight >= 3 kg

is required.

But satisfying weight alone isn't enough.

The products must also physically fit.

4. The biggest ambiguity: what does "most suitable" mean?

This is one of the most important engineering decisions in the assignment.

The document says the system should recommend the **most suitable box**, but it doesn't define a precise optimization criterion.

Assignment - Python_Django Deve...

Therefore, you should define your interpretation explicitly in your README/design.

Possible interpretations include:

Option A — Cheapest valid box

Among all boxes that can contain the order:

```
choose minimum cost
```

Example:

```
Small = ₹50 ✓
```

```
Medium = ₹80 ✓
```

```
Large = ₹150 ✓
```

Recommendation:

```
Small
```

Option B — Smallest suitable box

Choose the box with the smallest internal volume that can accommodate the order.

Option C — Cost + size optimization

You could define a scoring/optimization strategy.

However, introducing a complicated scoring system when the assignment doesn't require it can make the solution harder to justify.

What matters

Whatever rule you choose, **document it clearly and test it.**

For a hiring assignment, unexplained business logic is more problematic than choosing one reasonable interpretation and defending it.

5. Product model

At minimum, a product needs information representing:

```
name
length
width
height
weight
```

For example:

```
Product
-----
name: Laptop
length: 30
width: 20
height: 3
weight: 2
```

The assignment explicitly establishes that products have dimensions and weight.

Assignment - Python_Django Deve...

You should also decide:

- Units
- Whether dimensions can be zero
- Whether negative dimensions are allowed
- Precision of weight
- Precision of dimensions

These decisions should be consistent throughout the system.

6. Box model

A box needs:

```
name
internal_length
internal_width
internal_height
max_weight
cost
```

The important word here is **internal dimensions**.

The assignment specifically says boxes have **internal dimensions**, maximum weight capacity and cost.

Assignment - Python_Django Deve...

That means you should compare the product/order against the **usable internal space**, not external dimensions.

7. Order model

The assignment talks about a customer placing an order containing products.

Therefore, conceptually you need:

```
Order
|
+-- Product
+-- Product
+-- Product
```

In a relational database, this will usually involve something like:

```
Order
OrderItem
Product
```

rather than putting everything directly into the `Order` table.

For example:

```
Order #101

Laptop      × 1
Keyboard    × 1
Mouse       × 2
```

This lets you calculate the complete order.

8. Important business calculation: total weight

For an order:

```
total_weight =  
    product1.weight × quantity1  
    + product2.weight × quantity2  
    + ...
```

Example:

```
Laptop:    2 kg × 1 = 2 kg  
Keyboard:  1 kg × 1 = 1 kg  
Mouse:     0.2 kg × 2 = 0.4 kg
```

```
Total = 3.4 kg
```

Then a box with:

```
max_weight = 3 kg
```

must be rejected.

A box with:

```
max_weight = 5 kg
```

passes the weight constraint.

9. The harder problem: dimensions

This is where the assignment becomes more interesting.

Suppose:

```
Product:  
30 × 20 × 10
```

and:

Box:

$35 \times 25 \times 15$

It fits.

But consider:

Product:

$40 \times 20 \times 10$

Box:

$30 \times 25 \times 15$

It may still fit if the product can be rotated:

Product orientation:

$40 \times 20 \times 10$

$20 \times 40 \times 10$

$20 \times 10 \times 40$

$40 \times 10 \times 20$

$10 \times 20 \times 40$

$10 \times 40 \times 20$

Therefore, a robust solution should think about **orientation/permutation of dimensions**.

For a single rectangular product, you can compare sorted dimensions:

```
product = sorted([L, W, H])
```

```
box      = sorted([L, W, H])
```

and check:

```
product[i] <= box[i]
```

for each dimension.

But **multiple products in one box** are substantially more complicated.

10. Multiple products are not just a volume calculation

This is a major point.

Suppose:

Product A = $8 \times 8 \times 8$

Product B = $8 \times 8 \times 8$

and:

Box = $16 \times 8 \times 8$

Two products can potentially fit side-by-side.

But if you simply calculate:

A volume = 512

B volume = 512

Total = 1024

and compare:

Box volume = 1024

you have **not actually proven that they fit**.

Volume is necessary in some approaches, but it is not sufficient to solve 3D packing.

This distinction is exactly the sort of thing that can demonstrate your engineering understanding.

11. You need to define the packing scope

The assignment itself does not specify whether you need:

- One product per box
- Multiple products per box
- Arbitrary 3D packing
- Product rotation
- Splitting an order into multiple boxes
- Fragile-product constraints

- Empty-space minimization
- Stacking restrictions

Therefore, **do not silently invent a complex warehouse-packing system.**

Instead, your README should explain the scope you implemented.

For example:

The system evaluates whether an order can be packed into a single available box under the implemented dimension and weight constraints.

Then clearly explain your packing assumptions.

This makes the solution auditable.

12. What Django is expected to do

The document explicitly calls this a **Django-based system.**

Assignment - Python_Django Deve...

A sensible implementation could contain:

```
Django
├── Product
├── Box
├── Order
├── OrderItem
└── Box recommendation logic
```

Depending on your architecture, the recommendation algorithm could live in:

```
services/
```

rather than putting all business logic into:

```
views.py
```

For example:

```
orders/
  models.py
  views.py
```

```
serializers.py  
services.py  
tests.py
```

The exact architecture is not mandated by the assignment.

13. API vs UI

The assignment doesn't explicitly say:

Build a frontend.

It also doesn't explicitly say:

Build a REST API.

It simply says to build a Django-based system.

Therefore, either can be reasonable depending on your implementation.

Given your Django/DRF background, a clean API-based solution would be natural.

For example conceptually:

```
http
```

followed by:

```
http
```

returning something like:

```
{  
  "box": "Medium",  
  "cost": 80,  
  "reason": "Fits dimensions and weight constraints"  
}
```

But this exact API is **not specified by the assignment**. It is an implementation choice.

14. Validation matters

You should think about invalid input.

Examples:

Invalid product

```
length = -10
```

Should not be accepted.

Invalid box

```
max_weight = -5
```

Should not be accepted.

Invalid quantity

```
quantity = 0
```

Depending on your business rules, this should probably be rejected.

Impossible order

Suppose:

```
Product weight = 100 kg
```

but every box supports at most:

```
50 kg
```

Your system should not randomly select a box.

It should return an explicit failure such as:

```
No suitable box found.
```

15. What they specifically require regarding AI

This is extremely important.

The assignment **allows AI**.

Assignment - Python_Django Deve...

But they require an:

```
AI_USAGE.md
```

containing six things:

1. Which AI tools you used
2. Prompts you gave
3. Output you accepted
4. Output you rejected or modified
5. Mistakes AI made
6. How you verified the final code

Assignment - Python_Django Deve...

This means you should not write something vague like:

```
I used ChatGPT to help me code.
```

That doesn't satisfy the requested detail.

You need a traceable record.

16. Example structure of AI_USAGE.md

Conceptually:

```
# AI Usage

## Tools Used

- ChatGPT
- GitHub Copilot
```

```
## Prompt 1
```

```
...
```

```
## Output Accepted
```

```
...
```

```
## Output Modified
```

```
...
```

```
## Mistakes Found
```

```
...
```

```
## Verification
```

```
...
```

The important part is that this should be **truthful**.

Do not manufacture prompts or pretend AI did something it didn't do.

17. The most dangerous requirement: chat transcript

The assignment explicitly requires:

Export the chat transcript and attach and include it in your repository.

And then gives a very strong warning:

"Make sure that you don't generate this using AI."

It says the assignment will be rejected immediately if they see that AI was used to generate the transcript.

Assignment - Python_Django Deve...

This is different from `AI_USAGE.md`.

AI_USAGE.md

You can write a truthful explanation of your AI usage.

Chat transcript

They want the **actual exported conversation**, not an AI-generated reconstruction.

So you should use the actual export/download function of the AI tool you used and preserve the original transcript.

Do **not** ask AI to fabricate:

```
"Generate my ChatGPT conversation transcript."
```

That directly conflicts with their requirement.

18. The second major AI restriction: Learnings

They require:

```
What did you learn in this assignment?
```

But explicitly say:

"Again, not to use AI to generate an answer for this."

Assignment - Python_Django Deve...

So this is another place where you need to write the content yourself.

You can think about what you actually learned, but the final `LEARNINGS.md` should be **your own writing**, not AI-generated prose.

Given that you've said you already know many of the technologies involved, your learning doesn't have to pretend you learned basic Django.

It could instead focus on things genuinely learned during the assignment, such as:

- packing/constraint reasoning
- edge cases
- translating business requirements into models
- validating AI-generated code
- testing algorithmic logic
- making assumptions explicit
- trade-offs in box-selection algorithms

But **you should formulate the final answer yourself** because the assignment specifically prohibits AI generation for this deliverable.

19. Test cases are mandatory

The submission explicitly requires:

```
Test cases.
```

Assignment - Python_Django Deve...

You therefore shouldn't submit only code.

You should demonstrate that the system handles different scenarios.

A useful test matrix would include cases such as:

Scenario	Expected behavior
Product fits box	Box recommended
Product doesn't fit dimensions	Box rejected
Weight exceeds capacity	Box rejected
Multiple valid boxes	Selection rule applied
No valid box	Explicit failure
Product rotation required	Correct behavior according to your algorithm
Invalid dimensions	Validation error
Invalid weight	Validation error
Multiple products	Correct packing behavior
Multiple quantities	Correct calculation

The exact tests depend on the algorithm you implement.

20. Test run output is also mandatory

The assignment requires evidence that the tests actually ran.

It accepts either:

Option 1

A:

GitHub Actions link

Option 2

Terminal output pasted into:

TEST_OUTPUT.md

This is explicitly specified in the document.

Assignment - Python_Django Deve...

For example:

```
===== test session starts =====  
...  
20 passed  
===== 20 passed in ... =====
```

The important thing is that the output represents an **actual test run**, not something manually typed.

21. Required submission structure

The assignment ultimately expects these items:

```
Repository  
|  
├─ README.md  
├─ AI_USAGE.md  
├─ Chat transcript  
├─ Test cases  
├─ TEST_OUTPUT.md  
└─ Django project/code
```

The document's submission section explicitly lists:

1. GitHub repository link
2. README.md
3. AI_USAGE.md
4. Exported chat transcript
5. Your learning
6. Test cases
7. Test run output

Assignment - Python_Django Deve...

22. What README.md should accomplish

The assignment doesn't give a prescribed README structure, so this is an implementation recommendation rather than a direct requirement.

A strong README could explain:

```
# AI-Assisted Box Selection System
```

```
## Problem
```

```
## Solution
```

```
## Architecture
```

```
## Data Model
```

```
## Box Selection Algorithm
```

```
## Assumptions
```

```
## API / Usage
```

```
## Setup
```

```
## Running the Application
```

```
## Running Tests
```

```
## Test Results
```

```
## AI Usage
```

```
## Limitations
```

The most important sections are:

Algorithm

Explain exactly how you determine whether a box is suitable.

Assumptions

State things the assignment didn't define.

Limitations

Explain what your algorithm does **not** solve.

That is much better than pretending the solution handles every possible packing problem.

23. What they probably don't want

Based strictly on the assignment's requirements, avoid submitting a project where:

✗ The algorithm is unexplained

`Python`
is not sufficient.

Run

✗ AI_USAGE.md is fabricated

Don't invent prompts or mistakes.

✗ The transcript is AI-generated

This is explicitly prohibited.

Assignment - Python_Django Deve...

✗ LEARNINGS.md is AI-generated

Also explicitly prohibited.

Assignment - Python_Django Deve...

✗ Tests are superficial

For example:

Python

Run

doesn't demonstrate the business logic.

✗ Test output is fabricated

The output should come from an actual test execution.

✗ Business assumptions are hidden

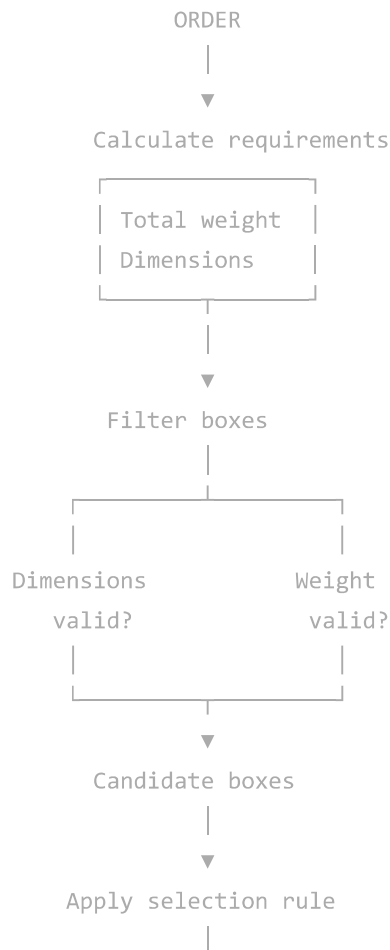
If "most suitable" means cheapest, smallest, or something else, document that.

24. What I would consider the core engineering challenge

The Django portion is relatively straightforward.

The interesting part is the **box-selection algorithm**.

You have three conceptual layers:



▼
Recommended box

That's the heart of the assignment.

25. The critical distinction: requirement vs implementation

This will help you avoid overengineering.

Explicitly required

From the document:

- Django-based system
- Product dimensions
- Product weight
- Box internal dimensions
- Box maximum weight
- Box cost
- Box recommendation
- AI usage documentation
- Actual chat transcript
- Your own learning
- Test cases
- Test execution output
- GitHub repository
- README

Assignment - Python_Django Deve...

Not explicitly specified

The document does **not** define:

- Exact database schema
- Exact API endpoints
- Frontend
- Django REST Framework
- Authentication
- Docker
- PostgreSQL
- Redis
- Celery

- AWS
- Exact packing algorithm
- Exact selection algorithm
- Exact UI
- Exact test framework

Therefore, don't add infrastructure merely because you can.

For a hiring assignment, a **small, understandable, well-tested implementation is generally easier to defend than unnecessary infrastructure.**

26. What your reviewer should be able to understand quickly

When they open your repository, they should be able to answer:

Problem

What does this application solve?

Data

What are Product, Order, OrderItem and Box?

Algorithm

How does the application determine whether a box is suitable?

Selection

If several boxes work, why is this one selected?

Edge cases

What happens when nothing fits?

Testing

How do I know the algorithm works?

AI

What did AI actually contribute?

Your contribution

What decisions did the candidate personally make?

That's the real objective behind the assignment's documentation requirements.

27. Your submission checklist

Before submitting, you should be able to check every item:

- ☐ Django project works from a clean setup
- ☐ Product model implemented
- ☐ Box model implemented
- ☐ Order/order-item handling implemented
- ☐ Dimension handling implemented
- ☐ Weight constraint implemented
- ☐ Box selection rule documented
- ☐ Impossible orders handled
- ☐ Validation implemented
- ☐ Test cases included
- ☐ Tests actually executed
- ☐ TEST_OUTPUT.md or GitHub Actions provided
- ☐ README.md included
- ☐ AI_USAGE.md included
- ☐ AI usage is truthful
- ☐ Actual AI chat transcript exported and included
- ☐ Chat transcript was **not generated by AI**
- ☐ Learning document written by **you**
- ☐ Learning document was **not generated by AI**
- ☐ No unexplained AI-generated code
- ☐ Repository link ready

The strongest focus should be on the **packing/selection logic, explicit assumptions, meaningful tests, and truthful AI documentation**. The assignment is short, but those are

the parts where a reviewer can distinguish a candidate who merely produced working code from one who understands the engineering problem.

Sources



the explanation was very detailed. I only used the tech stack,

the features and the project requirements.

Yes. If you **only used the provided tech stack, features, and project requirements** as the basis for understanding the assignment, then you don't need to overcomplicate it.

For your assignment, think of it simply as:

What you need to focus on

1. Tech stack

- Python
- Django
- Whatever additional technologies you actually used in the project

2. Required features

- Product management
- Box management
- Order handling
- Box recommendation/selection
- Relevant validations and tests

3. Project requirements

- Implement the requested Django system
- Follow the stated requirements
- Write tests
- Provide test output
- Provide `README.md`
- Provide `AI_USAGE.md`
- Include the actual chat transcript

- Include your own learning/ `LEARNINGS.md`

The important point is: **don't add requirements that aren't in the assignment just because they are technically possible.**

Since you've already built the project, the next useful step is to compare your existing `django-box-selector` project against the assignment **requirement-by-requirement** and identify:

-  completed
-  partially completed
-  missing
-  documentation still needed

That will tell you exactly what remains before submission.



ChatGPT can make mistakes. Check important info.